

Bezpečnost webových aplikací

Příprava k maturitě - SPŠT IT

Úvod

Bezpečnost webových aplikací je klíčovým aspektem vývoje a provozu moderních webových služeb. S rostoucí závislostí na internetu a webových aplikacích se zvyšuje i riziko útoků a zneužití těchto aplikací. Bezpečnost webových aplikací zahrnuje širokou škálu opatření a technik, které mají za cíl chránit data, uživatele a infrastrukturu před různými hrozbami, jako jsou SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF) a další. Implementace bezpečnostních opatření je nezbytná pro zajištění důvěry uživatelů a ochranu citlivých informací.

1 Bezpečnost webových aplikací

Webové aplikace jsou z důvodu jejich snadné dostupnosti častým cílem kybernetických útoků, které mohou způsobit škody velkého rozsahu od přerušení služeb až po krádež nebo ztrátu citlivých dat.

Velké množství aplikací však prochází fázemi rychlého vývoje s žádnou nebo extrémně krátkou dobou bezpečnostního (penetračního) testování. Čím více jsou webové aplikace komplexnější, tím se stává snaha o maximalizaci jejich zabezpečení časově náročnější, nákladnější a technicky složitější.

Bezpečnost webových aplikací by se neměla za žádných okolností podceňovat, neboť možný dopad úspěšného útoku může být devastující, často i zneužitím pouze jedné zranitelnosti přítomné ve zdrojovém kódu nebo konfiguraci aplikace.

Vektory útoků mohou být různé, například

- vstupy od uživatele,
- relace a cookies,
- chybná konfigurace,
- nezabezpečená komunikace,
- knihovny a moduly 3. stran.

Časté dopady útoků na webové aplikace

- znepřístupnění napadené služby nebo její podstatné části,
- odcizení, poškození, ztráta nebo modifikace citlivých dat,
- poškození pověsti subjektu provozující nebo vlastníci aplikaci,
- distribuce malware prostřednictvím napadené aplikace.

2 OWASP

OWASP neboli (Open Web Application Security Project) je projekt a otevřená komunita, která se zabývá bezpečností webových a mobilních aplikací.

OWASP otevřeně nabízí

- bezpečnostní nástroje a standardy,
- návody pro bezpečnostní testování,
- doporučení pro bezpečný vývoj,
- mezinárodní odborné konference,
- vzdělávací projekty a další materiály.

Všechny jejich nástroje, standardy a další dokumenty jsou dostupné zdarma všem. Nejznámějším dokumentem je žebříček OWASP Top 10 tento dokument obsahuje popis zranitelností a způsoby jejich zamezení, obsahuje celkem 10 nejkritičtějších webových zranitelností.

Seznam zranitelností z roku 2021

1. Broken Access Control,
2. Cryptographic Failures,
3. Injection,
4. Insecure Design,
5. Security Misconfiguration,
6. Vulnerable and Outdated Components,
7. Identification and Authentication Failures,
8. Software and Data Integrity Failures,
9. Security Logging and Monitoring Failures,
10. SSRF (Server Side Request Forgery).

3 Útoky na uživatele

Vektorem útoku na uživatele webových aplikací bývá nejčastěji injektovaný škodlivý kód, který je interpretován webovým prohlížečem klienta.

V kontextu webových aplikací se uvažují nejčastěji

- injekční útoky,
- reflektované útoky,
- útoky na relaci,
- distribuce malware.

3.1 XSS

XSS je injekční útok, který v případě napadení uživatele spočívá v interpretaci (překladi) škodlivého JavaScript kódu ve webovém prohlížeči klienta. Zranitelnosti na XSS obvykle umožňují útočníkovi provádět veškeré akce, které je uživatel schopen v dané relaci vykonat a číst data, ke kterým má přístup.

Dělení útoku

- Reflektovaný – XSS pochází z požadavku HTTP.
- Perzistentní – XSS je trvale uložen v databázi webové aplikace.
- DOM-based – XSS se nachází pouze na straně klienta (např. URL).

Pokud má napadený uživatel v rámci aplikace privilegovaný přístup, může být útočník schopen získat plnou nebo částečnou kontrolu nad funkcemi aplikace. Dopad útoku může mít pro uživatele destruktivní povahu od znefunkčnění celé aplikace až po malwarové útoky, které jsou distribuovány prostřednictvím XSS.

3.1.1 Příklady

Následující příklad se stal na Twitteru. Po načtení tohoto skriptu se automaticky sdílel.



Obrázek 1: „Ukázka XSS útoku na Twitteru“

3.1.2 Mitigace:

- Externí vstupy musí být před zpracováním validovány a sanitizovány.
- Pro všechny uživatelem ovlivnitelné parametry vypustit speciální znaky pomocí specifické escape syntaxe pro interpret programovacího jazyka.
- Použít standardizované kódování znaků v URL (tzv. percent encoding).
- Odesílat klientům pokyny pro zabezpečení jejich požadavků, např. přes bezpečnostní hlavičky (CSP, HSTS, X-Frame-Options atd.).
- Omezit ve zdrojovém kódu potenciálně nebezpečné reference, jako je např. innerHTML, a nahradit je bezpečnou variantou (textContent nebo value).
- Nasadit WAF (web application firewall) pro zajištění přidané bezpečnostní vrstvy v rámci komunikace mezi klientem a serverem.
- V případě PHP používat funkci htmlspecialchars(\$promenna);

3.2 CSRF

CSRF je zranitelnost webové aplikace, která umožňuje útočníkovi přimět uživatele k vykonání takové akce, kterou původně nechtějí provést. Zranitelnost umožňuje útočníkovi částečně obejít bezpečnostní ochranu, která je navržena pro kontrolu neautentizovaných a neautorizovaných uživatelů.

Při úspěšném útoku způsobí útočník, že jeho oběť provede akci neúmyslně

- Takovou akci může být např. změna e-mailové adresy na e-mail útočníka, změna hesla, převod finančních prostředků nebo odhlášení uživatele.
- Pokud má napadený uživatel v rámci webové aplikace privilegovanou roli, lze získat určitou kontrolu nad funkcemi prostředí, ve kterém se nachází.

3.2.1 Příklady

Útočník odešle oběti email obsahující odkaz na škodlivou webovou stránku, která vypadá identicky. Oběť vyplní požadovaná pole a po odeslání formuláře je přesměrován na legitimní

stránku, například API endpoint pro odesílání peněz. Legitimní stránka požadavek zpracuje a oběť přijde o peníze. Pokud by banka X používala CSRF token, akce by nebyla provedena, jelikož by hodnota uložena na serveru a hodnota odeslaná ve formuláři neodpovídala.

3.2.2 Mitigace

CSRF token – unikátní hodnota, kterou generuje aplikace na straně serveru a sdílí ji s klientem. Při pokusu o provedení určité akce (např. odeslání formuláře) musí klient do HTTP požadavku zahrnout správný token CSRF, bez kterého aplikace požadavek zamítne. To útočníkovi velmi ztěžuje sestavení platného požadavku jménem oběti.

Ověřování na základě odkazu – některé aplikace využívají hlavičku HTTP Referer k obraně proti útokům CSRF. Obvykle se ověřuje to, zda požadavek pochází z vlastní domény aplikace (méně účinné než CSRF tokeny).

4 Útoky na server

4.1 (D)DoS

4.2 SQL Injection

SQL injection je typ útoku, který napadá databázovou vrstvu vsunutím (odtud slovo injection) kódu přes neošetřený vstup. S pomocí takto vsunutého kódu může útočník získat citlivé osobní informace, jako například číslo kreditní karty nebo přihlašovací údaje. Také může databázi poškodit smazáním dat, dokonce může databázi upravit ve svůj prospěch.

4.2.1 Příklady

Aplikace používá nedůvěryhodná data během konstrukce následujícího zranitelného SQL dotazu: `String query = "SELECT * FROM users WHERE id='" + request.getParameter("id") + "'";`

Útočníkovi stačí pro injekční útok upravit id parametr v URL: `https://example.com/app/account-info?id=' or '1'='1' '`

4.2.2 Mitigace

Validace vstupů: zda vstupní data odpovídají očekávanému formátu a typu.

Parametrické dotazy: kde jsou data oddělené od SQL příkazu a bezpečně vloženy do něj.

Escapování speciálních znaků: neutralizace speciálních znaků ve vstupu, které by mohly být interpretovány jako část SQL syntax.

Web Application Firewall (WAF): který dokáže detekovat a blokovat podezřelé SQL dotazy.

4.3 SSRF

K SSRF (Server-Side Request Forgery) dochází v případech, kdy webová aplikace komunikuje se vzdáleným zdrojem bez dalšího ověření. Útočník tak dokáže aplikaci přimět k odeslání vytvořeného požadavku na vybraný cíl (často pod kontrolou útočníka).

4.3.1 Mitigace

Ověřovat a sanitizovat všechna vstupní data, Neodesílat klientům nezpracované odpovědi ze serveru. Nastavit bránu firewall ve výchozím stavu jako odepřít nebo nastavit pravidla řízení přístupu k síti tak, aby byl kromě nezbytného blokován veškerý provoz.

4.3.2 Příklady

Útočníci mohou získat přístup k lokálním souborům nebo interním službám a získat tak citlivé informace (např. `file: ///etc/passwd`, `http://localhost:2301`).

Útočník dokáže zneužít interní služby k dalším útokům, jako je vzdálené spuštění kódu (RCE - Remote Code Execution) nebo odepření služby DoS (Denial of Service).

Závěr

Shrnutí.

Zdroje

- Youtube - Téma
 - Wiki - Téma
-